Character Encoding & Charset

A foundational lecture on HTML character encoding, entities, URI encoding, frames, iframes, and security best practices.

What is Character Encoding?

Character encoding is a system used to represent characters as numbers so computers can store and display text correctly.

The Standard: UTF-8

UTF-8 is the most widely used character encoding for HTML. Always declare it in the document head:

```
<meta charset="UTF-8">
```

Why UTF-8?

Universal Language Support

Supports English and almost every major writing system on the planet.

Special Characters & Symbols

Handles special characters, symbols, and emoji without issues.

Prevents Display Errors

Helps prevent incorrectly displayed or garbled text across browsers.

Modern Standard

Recommended for all modern HTML documents by the W3C.

Full Example

```
<!DOCTYPE html>
<html>
<head>
 <meta charset="UTF-8">
 <title>Character Encoding</title>
</head>
<body>
 <p>Hello
```

What Are HTML Entities?

Definition

HTML entities are special codes used to represent characters that have special meaning in HTML or characters that are difficult to type directly.

Syntax

```
&entity_name;
&#entity_number;
```

Both forms are valid. Named entities are more readable; numeric entities cover characters without a named alias.

How Entities Work

When the browser parses HTML, it converts entity codes back into the intended character before rendering. This means you can safely include characters like `<` or `&` in your content without confusing the HTML parser.

Common HTML Entities Reference

Character	Entity	Meaning
<	`<`	Less than
>	`>`	Greater than
&	`&`	Ampersand
"	`"`	Double quotation
'	`'`	Apostrophe
©	`©`	Copyright
₹	`₹`	Indian Rupee

HTML Entities — In Practice

Code Example

```
<p>5 &lt; 10</p>
<p>Tom &amp; Jerry</p>
<p>&copy; 2026</p>
```

Rendered Output

5 < 10

Tom & Jerry

© 2026

Reading the Output

Each entity code is replaced by the browser with the correct character at render time. The HTML source remains safe and unambiguous — the parser never confuses `<` for the start of a tag, while the user sees the intended less-than sign on screen.

Why Do We Need HTML Entities?

The Core Problem

Some characters have special meanings in HTML. For example, the `<` character signals the beginning of an HTML tag. Writing it bare inside content causes the parser to misinterpret it.

Dangerous (broken markup)

```
<p>5 < 10</p>
```

Correct (using an entity)

```
<p>5 &lt; 10</p>
```

Frequently Used Entities

- `©` → ©
- `®` → ®
- `™` → ™
- ` ` → Non-breaking space
- `♥` → ❤️
- `✓` → ✓

Practical Example — Footer Markup

```
<footer>
  <p>&copy; 2026 My Website &trade;</p>
</footer>
```

Remember: Entities are especially useful when displaying characters that could otherwise be interpreted as HTML.

URI and URL Encoding

What is a URI?

URI — Uniform Resource Identifier. A URI identifies a resource on the internet or within a system. A URL is a type of URI that provides a way to *locate* a resource.

```
https://example.com/products?id=101
```

Why Encode a URI?

URLs can contain characters that have special meanings or aren't safe to use directly in a network request. URI encoding converts these characters into a safe percent-encoded format.

URI Encoding Reference

Character	Encoded
Space	`%20`
&	`%26`
?	`%3F`
#	`%23`
/	`%2F`
=	`%3D`

Simple Encoding Example

Plain text

```
Hello World
```

URI-encoded

```
Hello%20World
```

URI Encoding in HTML & JavaScript

Example with Query Parameters

Suppose we want to search for: **HTML & CSS**

An encoded URL used in an anchor tag:

```
<a href="https://example.com/search?q=HTML%20%26%20CSS">
  Search HTML & CSS
</a>
```

JavaScript Encoding Functions

JavaScript provides built-in functions for URI encoding and decoding:

```
encodeURIComponent("HTML & CSS");
// Result: HTML%20%26%20CSS

decodeURIComponent("HTML%20%26%20CSS");
// Result: HTML & CSS
```

Key Difference: HTML Entities vs. URI Encoding

HTML Entity Encoding

& → &

Used to safely represent reserved characters *inside HTML markup*. Parsed and replaced by the browser's HTML engine before rendering.

URI Encoding

& → %26

Used to safely transmit reserved characters *inside a URL*. Parsed and replaced by the browser's URL processor before the HTTP request is sent.

They solve different problems — never use one in place of the other.

Frames in HTML

What Are Frames? (Historical Context)

Historically, HTML provided `<frameset>` and `<frame>` to divide a browser window into multiple independent sections, each loading a separate HTML document.

```
<frameset cols="30%,70%">
  <frame src="menu.html">
  <frame src="content.html">
</frameset>
```

⊗ **Important:** `<frameset>` and `<frame>` are **obsolete** and must not be used in modern HTML.

Modern Alternative — CSS Layout

Replace frame-based layouts with semantic HTML and CSS Flexbox or Grid:

```
<div class="container">
  <aside>Sidebar</aside>
  <main>Content</main>
</div>
```

```
.container {
  display: flex;
}
```

Embedding Another Document

When you genuinely need to embed a separate document inside a page, the modern and supported approach is:

```
<iframe src="page.html"></iframe>
```

1

Old: <frameset>

Obsolete. Divides entire window. Not supported in HTML5.

2

Modern: CSS Flex/Grid

Full layout control. Semantic. Accessible. Responsive.

3

Embedding: <iframe>

Inline frame. Supported. Secure when configured correctly.

<iframe> and Security

<iframe> — Basic Usage

`<iframe>` allows external or internal content to be embedded inline inside a webpage.

```
<iframe
  src="https://example.com"
  title="Example Website">
</iframe>
```

Common Uses

- YouTube videos
- Google Maps
- Documents
- External applications
- Payment or authentication integrations

Security: The sandbox Attribute

Do not blindly embed content from untrusted websites. Use the `sandbox` attribute to restrict what the embedded content can do:

```
<iframe
  src="https://example.com"
  title="Embedded content"
  sandbox>
</iframe>
```

You can selectively restore capabilities as needed:

```
<iframe
  src="https://example.com"
  title="Embedded content"
  sandbox="allow-scripts">
</iframe>
```

> **Important:** Only grant the permissions the embedded content actually needs.

HTML Security Best Practices — Part 1

1 Avoid Untrusted HTML

Never directly insert untrusted user input into HTML.

```
//
```

HTML Security Best Practices — Part 2

1. Secure External Links

When opening an untrusted external website in a new tab, always include `rel="noopener noreferrer"`:

```
<a
  href="https://example.com"
  target="_blank"
  rel="noopener noreferrer">
  Visit Website
</a>
```

`noopener` prevents the new page from accessing the opener via `window.opener`, which could otherwise be exploited to redirect your page.

2. Use Safe Forms

Always use HTTPS when submitting sensitive information. Use `method="post"` for credentials — never GET, which appends data to the URL.

```
<form action="/login" method="post">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="submit">Login</button>
</form>
```

3. Use alt Text

Always provide meaningful `alt` attributes on images. This improves accessibility and provides fallback text when images cannot be displayed.

```
<img src="student.jpg"
   alt="Student studying HTML">
```

4. Use Security Headers

Configure your web server to send important HTTP security headers:

> **Content-Security-Policy (CSP)** — Controls which resources the browser may load.

> **Strict-Transport-Security (HSTS)** — Forces HTTPS connections.

> **X-Content-Type-Options** — Prevents MIME-type sniffing.

> **Important:** These are generally configured by the web server, not simply by adding HTML.

Final Recap

A concise reference covering every topic from this lecture.

Topic	Key Idea
`charset`	Defines character encoding for the document
UTF-8	Recommended modern encoding — declare with `<meta charset="UTF-8">`
HTML Entities	Represent special HTML characters safely (e.g. `<`, `&`)
URI Encoding	Makes URI components safe using percent-encoding (e.g. `%20`)
`<frame>`	Obsolete — do not use in modern HTML
`<frameset>`	Obsolete — replace with CSS Flexbox or Grid layouts
`<iframe>`	Embeds another document inline; supported in HTML5
`sandbox`	Restricts iframe capabilities — grant only what is needed
HTTPS	Encrypts communication between browser and server
`textContent`	Safer way to display untrusted text — avoids XSS via `innerHTML`
`rel="noopener"`	Helps protect new-tab links from `window.opener` exploitation

🧪 Lab Task

Create a webpage that demonstrates all of the concepts covered in this lecture. Your page must include each of the following elements:

1

UTF-8 Encoding

Declare UTF-8 character encoding using the correct `<meta>` tag in the document head.

2

HTML Entities

Use at least 10 distinct HTML entities visibly within your page content.

3

Encoded URL

Include a URL containing encoded query parameters (e.g. `%20`, `%26`).

4

<iframe> with sandbox

Embed an `<iframe>` with both a `title` attribute and the `sandbox` attribute configured appropriately.

5

Secure External Link

Add an external link using `target="_blank"` and `rel="noopener noreferrer"`.

6

POST Form

Include a form that submits data using the POST method.

7

Image with alt Text

Include at least one image with a meaningful, descriptive `alt` attribute.

8

Security Best Practices Section

Add a written section explaining 5 HTML security best practices in your own words.

🎯 Challenge Question

The Question

What is the difference between `<` and `%3C`?

The Answer

`<` is an **HTML entity** — it is processed by the browser's HTML parser and rendered as the `<` character inside page content.

`%3C` is **URI percent-encoding** — it is processed by the URL handler and represents the same `<` character when transmitted inside a URL.

Both encode the same underlying character, but they operate in different contexts and must never be used interchangeably.

<

HTML Entity
Context: inside HTML markup
Processed by: HTML parser
Renders as: <

%3C

URI Percent-Encoding
Context: inside a URL
Processed by: URL/HTTP layer
Represents: <

Different problems, different solutions — knowing which tool to use and when is the mark of a careful web developer.